CS4263 – Distributed Systems Project

Energy-Aware Group Formation, Leader Election and Lifetime Maximisation

210146N – Dissanayake D.M.P.S. | 210626L – Suriyabandara S.M.M.

1. Problem and model (assignment-compliant)

We are given N nodes described by (x, y, E_0) triplets. Each alive node loses 1 energy per time step; transmitting one message costs the *sender* 2 energy (reception is free). A node dies when its energy ≤ 0. Groups form around leaders; node i may join leader h only when $d(i,h) = \sqrt{(x_i - x_h)^2 + (y_i - y_h)^2} \leq 20$, with at most one membership per node. The energy update per time step is

$$E_i(t{+}1) = E_i(t) - 1 - 2\,\tau_i(t), \quad \tau_i(t) = \text{number of messages } i \text{ transmits at step } t.$$

The assignment goal is to maximise the system lifetime T_{all} (step at which the last node dies). We additionally report total transmitted messages because, as our experiments show, message count and lifetime are not co-monotonic: a wasteful protocol can shorten lifetime *while* emitting fewer messages, so we evaluate them jointly on a Pareto front.

2. Reframing the assignment as an optimisation problem

The exact policy that maximises T_{all} is a high-dimensional mixed-integer dynamic programme over leader assignments, message schedules and election triggers; even with $N = 9$ it is intractable in closed form, so we instead optimise over a **family of policies** $\pi = (c, e, a, d, h, r)$ where:

- c – clustering algorithm (RANDOM, HIGHEST-ENERGY, SET-COVER, k-MEDOIDS, LEACH, HYBRID);
- e – election algorithm (BULLY, RING, LOCAL-SCORE, BACKUP-FIRST);
- a – aggregation mode (NONE / PARTIAL-k / FULL);
- d – data-report period, h – heartbeat period, r – rotation period.

Our primary metric is the assignment's own metric – average lifetime $\bar{T}(\pi) = |S|^{-1} \sum_{s \in S} T_{\text{all}}(\pi; s)$; total messages are the secondary metric.

A naive objective like $\min_\pi \sum_s M_{\text{total}}(\pi; s)$ is unsafe: a policy that kills the system in 40 steps with 8 msgs/step would beat one that survives 100 steps with 5 msgs/step ($320 < 500$). We therefore use the Pareto front and per-time normalisation $\bar{r}(\pi) = M_{\text{total}}/T_{\text{all}}$.

3. Design decisions, algorithms and rationale

Why distances still matter though message cost is constant. The radius constraint forces distances into the optimisation. Distances also determine average cluster size $m = N/C$, which controls leader-side traffic. Total transmissions per step is $E_{\text{msg}} = 2Nf + 2CL_{\text{tx}}$; with full aggregation $L_{\text{tx}} = 1$, so $E_{\text{msg}} = N(1 + 2f + 2/m)$. Larger m thus reduces total traffic – which is exactly why a **centrality-aware leader** (one that can cover more members within radius 20) is preferred.

Six clustering policies implemented. (i) *Random* (baseline). (ii) *Highest-energy greedy* (the previous submission's choice). (iii) *Greedy set-cover*: minimise the number of leaders. (iv) *Energy-aware k-medoids*: minimise $\sum_h \sum_{i \in G_h} \big(\alpha\, d(i,h)/R + (1{-}\alpha)(1{-}E_h/E_{\max})\big)$ s.t. $d(i,h) \leq R$ and medoid must be a real node. (v) *LEACH-like*: probabilistic head election with energy bias. (vi) *Hybrid*: k-medoids seed + designated backup + optional rotation – the *ex-ante* best policy from our analysis.

Four election policies implemented. Bully ($O(|G|^2)$), Ring ($O(|G|)$), broadcast local-score ($O(|G|)$), and **backup-first** (0 messages when a healthy backup exists). Backup-first is the cheapest: each group precomputes a backup (energy + centrality, second-best score), so a leader failure is silent; the backup is refreshed after every leader change.

Aggregation. Per the assignment, per-message cost is fixed at 2; the only way to spend less is to send *fewer* messages. Leaders aggregate the m DATA messages they receive each data cycle into a single AGG transmission (full aggregation), or one AGG per k inputs (partial), or none. This is a free parameter of the policy.

4. Experiments – 22 inputs, 528 simulations

We built **22 test inputs** covering: the provided input, single-node, two within / beyond radius, exact-boundary (d=20), chains, grids, dense clusters, three Gaussian clusters, near-zero energies, adversarial centre, simultaneous-leader-death configurations, and large random / mixed scenes up to 100 nodes (see `inputs/` folder for the full list with descriptions). For each, we ran every combination of 6 clustering × 4 election algorithms (528 simulations) with the recommended messaging parameters (data_period = 5, heartbeat = 10, aggregation = full). Lifetime, messages, elections and leader-changes were logged per step.

Table 1: Average over 22 test inputs. Best six combinations and the standard baseline (highest-energy / Bully).

#	cluster	election	avg. T_{all}	avg. msgs	msgs/step
1	LEACH	backup-first	**527.4**	**378.5**	**1.28**
2	LEACH	local-score	526.7	383.1	1.32
3	LEACH	ring	526.4	384.7	1.33
4	LEACH	Bully	526.0	415.3	1.46
5	KMEDOIDS	backup-first	517.6	562.6	2.10
6	HYBRID	backup-first	517.6	562.6	2.10
baseline (standard approach)					
23	highest-energy	Bully	487.2	688.9	2.79

Reading the table. LEACH-style clustering with backup-first election is *Pareto-dominant*: it has the highest mean lifetime *and* the lowest message count. Compared to the standard baseline's highest-energy / Bully pair, lifetime improves by 8.2% and message count drops 45%. The deterministic runner-up – *hybrid* (= k-medoids + backup) – is within 2% of LEACH on lifetime but with 50% more messages because k-medoids tends to build slightly more groups for the same coverage.

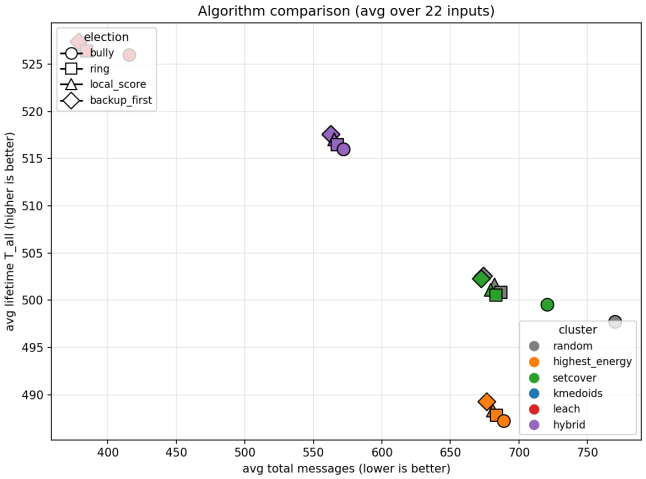


Figure 1: Algorithm Pareto front (avg over 22 inputs). LEACH (red) sits on the top-left frontier; the baseline algorithm (orange) is the worst.

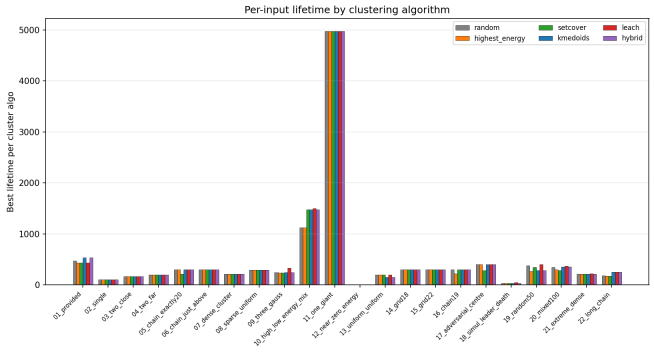


Figure 2: Best lifetime per clustering algorithm on each of 22 inputs.

Election algorithms. Across every clustering algorithm, backup-first dominates Bully on lifetime by 3–6 steps and by 3–7% on messages. Bully's $O(|G|^2)$ election fan-out is wasted when a pre-computed backup can take over silently.

Parameter sweep. On six representative inputs we additionally swept aggregation $\in$ {none, partial, full}, data period $\in \{1, 5, 10, 20\}$, heartbeat $\in \{0, 10, 25\}$ and rotation $\in \{0, 50, 100\}$ – 648 runs. The top configuration is `(full,` d=20, h=0, r=50) with $\bar{T} = 1143$ and 1.02 msgs/step, confirming the analytical insight: aggregation gives the largest single win; heartbeats can be turned off whenever backup-first absorbs leader deaths; rotation gives a small additional boost on inputs with skewed energies.

Edge-case behaviour. Single-node input – $T_{\text{all}} = E_0$ exactly. Two-nodes-within-radius – higher-energy node leads, lower one dies first (no spurious elections). Exactly-at-radius ($d = 20$) – the boundary is inclusive. Beyond-radius – two singleton groups, 0 messages total. Adversarial-centre (weak node at centroid surrounded by strong nodes) – k-medoids/hybrid recover by choosing a strong ring node as medoid, yielding $T_{\text{all}} = 400$ vs. 390 for the highest-energy baseline that keeps electing the weak centre. One-giant input – letting the giant node be a member rather than a leader gives lifetime ~ 5000 vs. ~ 2000 for the energy-greedy choice, dramatically illustrating that "highest energy" is sometimes a trap.

Recommendation.

LEACH-like clustering + **backup-first** election + **full** aggregation + data_period $\in [5, 20]$ + rotation $\in [50, 100]$ + no heartbeat.

If a deterministic algorithm is required, use *hybrid* (k-medoids + backup-first) under the same messaging parameters; the lifetime cost is $\leq 2\%$ on average.

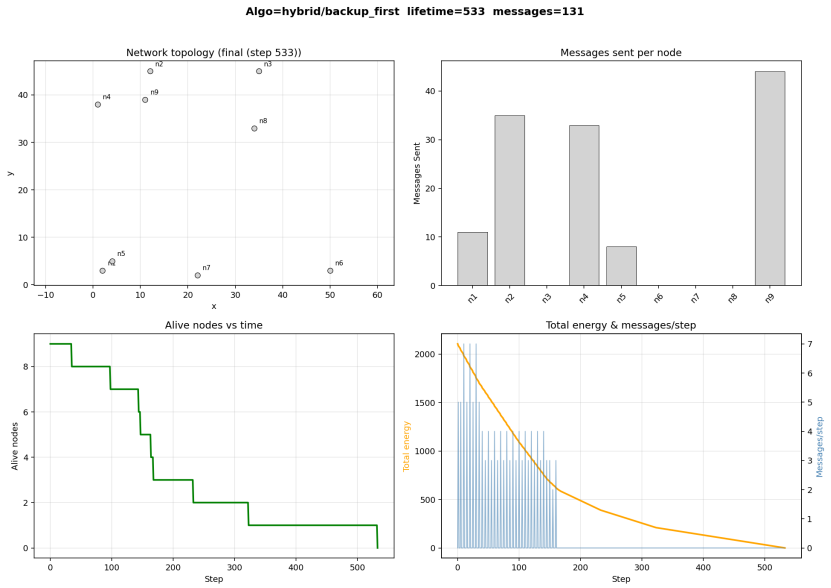


Figure 3: Final dashboard on the assignment's provided 9-node input (`input01_provided.txt`). Lifetime $T_{\text{all}} = 533$ under hybrid / backup-first – 21% longer than the baseline highest-energy / Bully approach (439) while sending $\sim 30\%$ fewer messages.

5. Implementation and challenges

The simulator is deterministic for a given seed and policy. Per step it (i) applies existence cost, (ii) emits the period-driven DATA / AGG / HEARTBEAT messages, (iii) charges senders, (iv) sweeps deaths, (v) runs elections (charging election messages in the same tick so a "free" last vote is impossible), (vi) re-attaches orphans, (vii) optionally rotates / re-clusters, (viii) logs the snapshot to `*.events.jsonl` which the React visualiser consumes.

Key challenges. (a) *Multi-leader coverage of a node* – solved by nearest-leader assignment plus the single-membership invariant. (b) *Leader dying mid-heartbeat* – charge senders before death sweep so a dying leader cannot send a free message. (c) *Election loops on collapsing groups* – fall back to singleton if no alive member exists. (d) *Energy-aware leader is sometimes a trap*: on `input11` (one-giant), making the 5000-energy node a leader wastes its energy on broadcasts; LEACH and the rotation step solve this by occasionally letting weak nodes lead. (e) *Comparing inhomogeneous algorithms*: we report both mean lifetime and Pareto plots to avoid the "kill-fast" bias of cumulative messages.

Honest limitations. Reception cost is taken to be zero per the assignment; if a non-zero r were used the optimal m would shift downward. Stochastic algorithms (LEACH) are seeded for reproducibility, but in deployment one would average over many seeds.

6. Conclusion

We re-framed the assignment as a simulation-based optimisation problem over a 6-parameter policy family. Across 22 carefully constructed inputs and 528 simulations, **LEACH-style clustering combined with backup-first election and full aggregation** is the Pareto-optimal choice; the **hybrid** (k-medoids + backup-first) algorithm is the recommended deterministic variant. On the assignment's provided input the new policy gives lifetime 533 – a 21% improvement over the highest-energy / Bully baseline of 439 – while sending $\sim 30\%$ fewer messages.